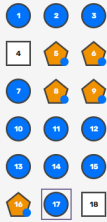


## Instructions



A gym tracks, for each piece of equipment, how many people are currently using it. Through the course of an hour this changes, more people join an existing piece of equipment, a brand-new piece of equipment gets tracked for the first time, and one piece of equipment is taken offline for maintenance and stops being tracked.

Write a program that tracks this, applies each event below in the exact order described, and then reports on the current state.

Do the following, in order:

1. Read **three pieces of equipment**, each as a **name** (line), then its current number of **users** (a whole number). Print a status line for each of the three, in the order entered.
2. **Booking update** – read an equipment **name** (guaranteed to be one of the three above) and an **amount**, and add that amount to that equipment's current user count. Print the updated status of that equipment.
3. **New equipment** – read a **name** and a starting user count, and start tracking it. Print its status.
4. **Equipment removed** – read an equipment **name** (guaranteed to currently be tracked). Stop tracking it, and print the user count it had at the moment it was removed.
5. **Query** – read an equipment **name** (guaranteed to still be tracked at this point). Print its current status.
6. **Report** two final values, using only the equipment still being tracked at this point: the **total number of users** across all of them, and the name and user count of the **busiest** equipment (the one with the highest user count – there will be no tie for the highest among the equipment tracked at this point in the tests).

Input Format (11 lines, in order)

- Lines 1-6: three pieces of equipment – name1, users1, name2, users2, name3, users3
- Lines 7-8: booking update – an equipment name (one of the three above) and an amount to add
- Lines 9-10: new equipment – a name and a starting user count
- Line 11: the name of the equipment to remove
- Line 12: the name to query

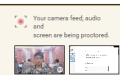
Output Format

```
<name1>: <users1> users
<name2>: <users2> users
<name3>: <users3> users
Updated <booking name>: <new users> users
Added <new name>: <its users> users
Removed <removed name> with <its final users> users
Usage of <query name>: <its users> users
Total users: <sum>
Busiest equipment: <name> (<users>)
```

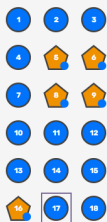
Examples

< Previous

Next >



## Instructions



## The Gym Equipment Tracker

A gym tracks, for each piece of equipment, how many people are currently using it. Through the course of an hour this changes, more people join an existing piece of equipment, a brand-new piece of equipment gets tracked for the first time, and one piece of equipment is taken offline for maintenance and stops being tracked.

Write a program that tracks this, applies each event below in the exact order described, and then reports on the current state.

Do the following, in order:

1. Read **three pieces of equipment**, each as a **name** (line), then its current number of **users** (a whole number). Print a status line for each of the three, in the order entered.
2. **Booking update** – read an equipment **name** (guaranteed to be one of the three above) and an **amount**, and add that amount to that equipment's current user count. Print the updated status of that equipment.
3. **New equipment** – read a **name** and a starting user count, and start tracking it. Print its status.
4. **Equipment removed** – read an equipment **name** (guaranteed to currently be tracked). Stop tracking it, and print the user count it had at the moment it was removed.
5. **Query** – read an equipment **name** (guaranteed to still be tracked at this point). Print its current status.
6. **Report** two final values, using only the equipment still being tracked at this point: the **total number of users** across all of them, and the name and user count of the **busiest** equipment (the one with the highest user count – there will be no tie for the highest among the equipment tracked at this point in the tests).

Input Format (11 lines, in order)

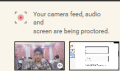
- Lines 1-6: three pieces of equipment – name1, users1, name2, users2, name3, users3
- Lines 7-8: booking update – an equipment name (one of the three above) and an amount to add
- Lines 9-10: new equipment – a name and a starting user count
- Line 11: the name of the equipment to remove
- Line 12: the name to query

Output Format

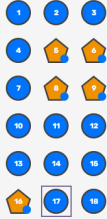
```
<name1>: <users1> users
<name2>: <users2> users
<name3>: <users3> users
Updated <booking name>: <new users> users
Added <new name>: <its users> users
Removed <removed name> with <its final users> users
Usage of <query name>: <its users> users
Total users: <sum>
Busiest equipment: <name> (<users>)
```

< Previous

Next >



## Instructions



## Example 1

## Input

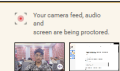
```
Treadmill
5
Rowing Machine
3
Bench Press
2
Treadmill
4
Elliptical
6
Bench Press
Rowing Machine
```

## Output

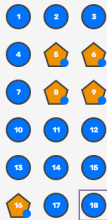
```
Treadmill: 5 users
Rowing Machine: 3 users
Bench Press: 2 users
Updated Treadmill: 9 users
Added Elliptical: 6 users
Removed Bench Press with 2 users
Usage of Rowing Machine: 3 users
Total users: 18
Busiest equipment: Treadmill (9)
```

## Explanation

- The three pieces of equipment are recorded and printed with their user counts as entered.
- The booking update names `Treadmill` with an amount of `4` this adds to Treadmill's existing 5, giving 9 (not replacing it).
- `Elliptical` is a brand-new piece of equipment starting at 6 users.
- `Bench Press` is removed; its user count at the moment of removal (2) is reported, and it is no longer tracked from this point on.
- The query is for `Rowing Machine` which is still tracked and unaffected by any of the previous events – its status (3 users) is printed as-is.
- At this point the tracked equipment is `Treadmill` (9), `Rowing Machine` (3), `Elliptical` (6) – total users =  $9+3+6 = 18$ . The busiest is `Treadmill` at 9.

[< Previous](#)[Next >](#)

## Instructions



## The Runway Queue

An airport control tower keeps a running order of aircraft call signs waiting for runway clearance. Through the course of a shift this order changes, a new arrival joins the back of the queue, and a diverted aircraft is pulled out of the queue entirely.

Write a program that builds this queue, applies both changes, and then reports a few simple facts about the queue as it stands afterwards.

Do the following, in order:

- Read five call signs (one per line) and record them as the initial queue, in the order they were entered.
- Read one more call sign – a new arrival. Add it to the **back** of the queue.
- Read one more call sign – an aircraft that has been diverted to another airport. Remove the **first occurrence** of that call sign from the queue entirely.
- Read one more call sign. Report **how many times** that exact call sign currently appears in the queue (a call sign can legitimately appear more than once, if the same aircraft cycles through more than once in a shift).
- Report the call sign currently at the **very front** of the queue, the call sign currently at the **very back**, and the total number of aircraft now in the queue.

Print exactly six lines, in the format shown below.

## Input Format

- Lines 1–5: the five initial call signs, one per line
- Line 6: the new arrival's call sign
- Line 7: the call sign to remove (the diverted aircraft, guaranteed to be present in the queue at that point; if it happens to occur more than once, remove only the first occurrence)
- Line 8: the call sign to count occurrences of (guaranteed to be present in the queue at that point at least once)

## Output Format

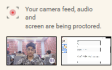
Exactly six lines, in this exact order:

```
Initial queue: <call sign>, <call sign>, ..., <call sign>
After adding arrival call sign: <call sign>, <call sign>, ...
After removing removed call sign: <call sign>, <call sign>, ...
Occurrences of <counted call sign>: <count>
First: <call sign>, Last: <call sign>
Total aircraft: <count>
```

Every queue line is the call signs joined with `" , "` (comma-space), with no surrounding brackets or quotes.

## Examples

[< Previous](#)[Next >](#)



Instructions



Input

```
AI181
UA282
SQ383
EK484
QF585
AI181
EK484
AI181
```

Output

```
Initial queue: AI181, UA282, SQ383, EK484, QF585
After adding AI181: AI181, UA282, SQ383, EK484, QF585, AI181
After removing EK484: AI181, UA282, SQ383, QF585, AI181
Occurrences of AI181: 2
First: AI181, Last: AI181
Total aircraft: 5
```

Explanation

- The five initial call signs are recorded in the order they were entered.
- The new arrival `AI181` is added to the back, note it is the same call sign as the very first entry, so `AI181` now appears twice. This is allowed.
- The first occurrence of `EK484` is removed. There was only one, so it's simply dropped, leaving five call signs.
- `AI181` now occurs twice in the queue (the original entry and the new arrival).
- The front of the queue is `AI181` (unchanged from the start); the back is also `AI181` (the aircraft that just arrived), two distinct aircraft entries that happen to share a call sign.
- Five aircraft remain in total: 5 initial + 1 arrival - 1 removed = 5.

Example 2

Input

```
DL111
AA222
BA333
```

[Previous](#)

[Next](#)